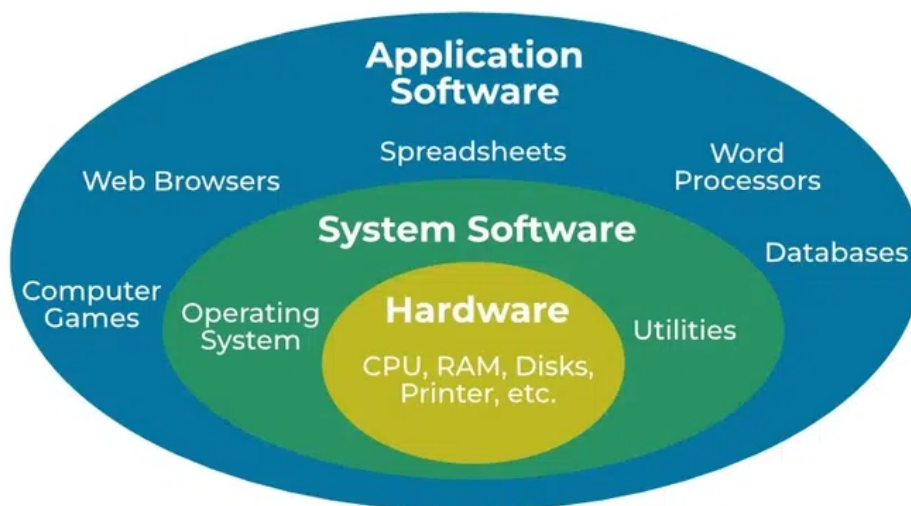


Introduction to Machine Architecture

System Software

- **System software** is software intended to support the operation and use of the computer itself, rather than any particular application.
- It is usually related to the architecture of the machine on which it runs (machine-dependent).
- Examples include assemblers, compilers, and operating systems, which manage resources like registers, instruction formats, addressing modes, and nearly all resources of a computing system.



Application Software	System Software
Focuses on solving a user's problem (e.g., word processor, game)	Focuses on operating and managing the computer itself
Uses computer as a <i>tool</i>	Supports the computer's own operation
Mostly machine-independent (if written in high-level language)	Mostly machine-dependent

Machine Dependency:

Software or features that **depend directly on the specific hardware architecture** of a computer (e.g., CPU type, instruction set, registers, addressing modes).

Examples:

- **Assemblers:** must know instruction formats and addressing modes.
- **Compilers:** must generate machine language code based on the number and type of registers and available instructions.
- **Operating systems:** manage nearly all hardware resources.

Machine Independency

Definition:

Software or features that **do not directly depend on the type of computing system** being used. They work the same way across different machines.

Examples:

- **General design and logic of an assembler** – basically the same on most computers.
- **Some compiler code optimization techniques** – independent of the target machine.
- **The process of linking together independently assembled subprograms** – usually does not depend on the computer.

Is Assembler machine dependent or machine independent?

An assembler has **both** machine-dependent and machine-independent aspects:

Machine-Dependent Parts (Mostly)

- Translates mnemonic instructions(LDA, STA) into **machine code**.
- Must know **instruction formats, addressing modes, registers** of the specific computer.
- *instruction formats, addressing modes, etc., are of direct concern in assembler design".* This makes the assembler **machine-dependent**.

Machine-Independent Parts

- The **general design and logic** of an assembler is basically the same on most computers.
- So the *conceptual structure* of an assembler is machine-independent.

Machine Architecture

Machine architecture is the **design and structure of a computer's hardware** that is visible to system software. It includes things like:

- Instruction formats (how commands are encoded)
- Addressing modes (how the CPU finds data in memory)
- Number and types of registers (small, fast storage inside the CPU)
- Available machine instructions (what operations the CPU can do)
- Memory organization and management

The **SIC (Simplified Instructional Computer)** is an example of a *hypothetical* machine architecture

Simplified Instructional Computer (SIC)

SIC is a **hypothetical computer** carefully designed to include the hardware features most often found on real machines, while avoiding unusual or irrelevant complexities. It exists in two upward-compatible versions:

- **Standard SIC** – the base model.
- **SIC/XE** – an extended version (“extra equipment”).

Its purpose is to separate **fundamental concepts** of system software from **machine-specific implementation details**, allowing the core ideas

of assemblers, compilers, and operating systems to be taught without the idiosyncrasies of any real computer.

SIC Machine Architecture

Memory

- **Byte-addressable:** 8-bit bytes.
- **Word:** 3 consecutive bytes = 24 bits.
- **Total size:** $2^{15}=32,768$ bytes. (In standard SIC, the **address field** in an instruction is only **15 bits** wide)
- Word address = address of its lowest-numbered byte.

Registers

There are **5 registers** all of which has special uses.

Each register is 24 bits in length (1 word).

Mnemonic	Number	Special use
A	0	Accumulator; used for arithmetic operations
X	1	Index register; used for addressing
L	2	Linkage register; the Jump to Subroutine (JSUB) instruction stores the return address in this register
PC	8	Program counter; contains the address of the next instruction to be fetched for execution
SW	9	Status word; contains a variety of information, including a Condition Code (CC)

Data Formats

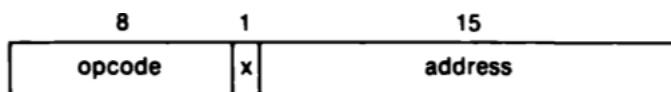
Integers are stored as 24 bits binary numbers.

2 complete representation is used for negative values

Characters are stored using 8 bit ascii codes.

Instruction Format

Every machine instruction on standard SIC is exactly **24 bits** (one word), divided into three fields:



The bit x is used to indicate indexed addressing mode.

Bits	Field	Width	Purpose
0-7	opcode	8 bits	Specifies the operation (e.g., LDA , ADD , JSUB).
8	x (flag)	1 bit	Addressing mode selector: 0 = direct, 1 = indexed.
9-23	address	15 bits	Provides the base address (or offset) for target address calculation.

5. Addressing Modes

Only two modes, selected by the x bit in the instruction.

Mode	x bit	Target Address (TA) Calculation	Effective Address Formula
Direct	0	TA = address	The 15-bit address field is the actual memory address.
Indexed	1	TA = address + (X)	Add the contents of index register X to the address field.

Instruction Set

Instruction Set Table for Standard SIC

Mnemonic	Operand	Opcode (hex)	Meaning (Effect)
Data Movement (Word)			
LDA	M	00	Load word from memory address M into register A. A ← (M)
LDX	M	04	Load word from M into register X. X ← (M)
LDL	M	08	Load word from M into register L (linkage). L ← (M)
STA	M	0C	Store word from register A into memory M. (M) ← A
STX	M	10	Store word from register X into memory M. (M) ← X
STL	M	14	Store word from register L into memory M. (M) ← L
STSW	M	E8	Store status word (register SW) into memory M. (M) ← SW (mainly the cor ◯ n code)

Byte Data Movement

LDCH	M	50	Load character (byte) from memory M into the rightmost 8 bits of register A. Other bits of A are unchanged. $A[7:0] \leftarrow (M)[7:0]$
STCH	M	54	Store character (byte) from the rightmost 8 bits of register A into memory M. $(M)[7:0] \leftarrow A[7:0]$

Arithmetic

ADD	M	18	Add word from memory M to register A. $A \leftarrow (A) + (M)$
SUB	M	1C	Subtract word from memory M from register A. $A \leftarrow (A) - (M)$
MUL	M	20	Multiply register A by word from memory M. $A \leftarrow (A) \times (M)$
DIV	M	24	Divide register A by word from memory M. $A \leftarrow (A) \div (M)$ (integer division)

Subroutine Calls

JSUB	M	48	Jump to subroutine . Store the return address (current PC + instruction length) into register L, then jump to M. $L \leftarrow PC; PC \leftarrow M$
RSUB	(none)	4C	Return from subroutine . Jump to the address stored in register L. $PC \leftarrow (L)$

Loop Control

TIX	M	2C	Test and increment x. Add 1 to register X, then compare the new X with the word at memory M (like COMP). Sets CC. $X \leftarrow X + 1; CC \leftarrow (X) : (M)$
-----	---	----	--

Input / Output (Programmed I/O)

TD	M	E0	Test device. The operand <code>M</code> is a one-byte device code (e.g., <code>X'F1'</code>). Sets CC: <code><</code> = device ready, <code>=</code> = not ready. <pre>CC ← device_status(M)</pre>
RD	M	D8	Read data from device <code>M</code>. Reads one byte into the rightmost byte of register A. <pre>A[7:0] ← data_from_device(M)</pre>
WD	M	DC	Write data to device <code>M</code>. Writes the rightmost byte of register A to the device. <pre>device(M) ← A[7:0]</pre>

Branching	J	M	3C	Unconditional Jump: Load the new value <code>M</code> into the Program Counter (PC). <pre>PC ← M</pre>
	JEQ	M	30	Jump if Equal: Load <code>M</code> into PC if the CC is set to <code>=</code>. <pre>if CC = '=', PC ← M</pre>
	JGT	M	34	Jump if Greater Than: Load <code>M</code> into PC if the CC is set to <code>></code>. <pre>if CC = '>', PC ← M</pre>
	JLT	M	38	Jump if Less Than: Load <code>M</code> into PC if the CC is set to <code><</code>. <pre>if CC = '<', PC ← M</pre>

Example

Adding two numbers and storing the result:

assembly

```
LDA    ALPHA    ; A ← value at ALPHA
ADD    BETA     ; A ← A + value at BETA
STA    GAMMA    ; store result into GAMMA
...
ALPHA  WORD     10
BETA   WORD     20
GAMMA  RESW     1
```

- LDA, ADD, STA are **opcode mnemonics** (the human-readable names for machine instructions).
- ALPHA, BETA, GAMMA are **labels** (also called **symbolic addresses** or **symbols**). They represent memory locations where data is stored.
- The directives WORD and RESW are **assembler directives** (pseudo-ops) that define storage.

Data Movement

Mnemonic	Operation	Description
LDA	Load A	Load word from memory into register A
LDX	Load X	Load word from memory into register X
STA	Store A	Store word from register A into memory
STX	Store X	Store word from register X into memory

Comparison and Conditional Branching

Mnemonic	Operation	Description
COMP	Compare	Compare (A) with (mem); set Condition Code (CC) to < , = , or >
JLT	Jump if less	Jump to address if CC is <
JEQ	Jump if equal	Jump to address if CC is =
JGT	Jump if greater	Jump to address if CC is >

Subroutine Calls

Mnemonic	Operation	Description
JSUB	Jump to subroutine	Store the return address (next instruction's address) into register L, then jump to the given address
RSUB	Return from subroutine	Jump to the address stored in register L

Input/Output (Programmed I/O)

Standard SIC uses **extremely simple I/O**: each byte is transferred one at a time **through register A's low 8 bits**.

I/O Instructions

Instruction	Operand	Effect
TD (Test Device)	8-bit device code	Tests if the device is ready to send or receive. Sets Condition Code: < = ready, = = not ready.
RD (Read Data)	8-bit device code	Reads one byte from the device into the rightmost 8 bits of register A. (High 16 bits of A are cleared? Not specified; likely unchanged or zeroed.)
WD (Write Data)	8-bit device code	Writes the low 8 bits of register A to the device.

To read a character from a keyboard (device code **0x01**):

```
LOOP  TD 0x01      ; test if keyboard is ready
      JEQ LOOP      ; if not ready (CC = =), loop and test again
      RD 0x01      ; ready (CC = <), read byte into A
      ...          ; now the character is in low byte of A
```

To write a character:

```
LOOP  TD 0x02      ; test if printer/display is ready
      JEQ LOOP
      WD 0x02      ; write byte from A
```

SIC/XE Machine Architecture

SIC/XE (Simplified Instructional Computer with **Extra Equipment**) is an upward-compatible extension of standard SIC. It adds:

- Larger memory (1 MB vs. 32 KB)
- More registers (B, S, T, F)
- Floating-point data type (48 bits)
- Multiple instruction formats (1, 2, 3, 4)
- New addressing modes: base relative, PC-relative, immediate, indirect
- I/O channels for concurrent I/O
- Additional instructions (floating-point arithmetic, register-to-register ops, supervisor call)

Memory

Feature	Specification
Size	$2^{20} = 1,048,576$ bytes (1 MB)
Organization	Same as standard SIC: byte-addressable, 3 bytes = 24-bit word
Address width	20 bits (0 to 1,048,575)

The 15-bit address field of standard SIC is insufficient. SIC/XE introduces new formats (3 and 4) with 12-bit displacement or 20-bit direct address, plus relative addressing.

Registers

Standard SIC registers (preserved)

A, X, L, PC, SW (as before, all 24 bits except F which is 48 bits)

New registers in SIC/XE

Mnemonic	Number	Size	Special Use
B	3	24 bits	Base register – used for base relative addressing ($TA = (B) + disp$)
S	4	24 bits	General working register – no special use
T	5	24 bits	General working register – no special use
F	6	48 bits	Floating-point accumulator – holds 48-bit float values for ADDF, SUBF, MULF, DIVF

Data Formats

Preserved from standard SIC

- Integers: 24-bit, 2's complement.
- Characters: 8-bit ASCII.



New: Floating-point (48 bits)

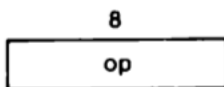
Bit layout:

Field	Width	Position (bits)	Meaning
s (sign)	1	0 (most significant)	0 = positive, 1 = negative
exponent	11	bits 1-11	unsigned integer (0-2047)
fraction	36	bits 12-47	binary fraction with implied binary point before the high-order bit

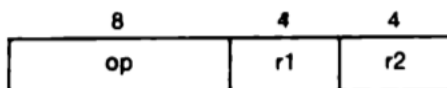
Instruction Formats

SIC/XE has **four formats** to support different operand types and addressing needs.

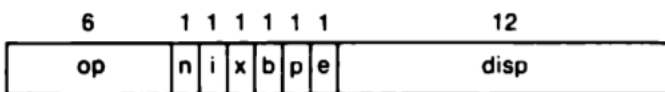
Format 1 (1 byte):



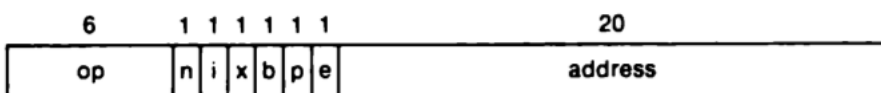
Format 2 (2 bytes):



Format 3 (3 bytes):



Format 4 (4 bytes):



Format 1 (1 byte, no operands)

| opcode (8 bits) |

- Used for instructions that need no operands (e.g., RSUB actually RSUB in standard SIC was 3 bytes; in SIC/XE some instructions like HIO, TIO, SIO might be Format 1. Appendix A gives specifics.)

Format 2 (2 bytes) – register-to-register operations

| opcode (8 bits) | r1 (4 bits) | r2 (4 bits) |

- r1 and r2 are register numbers (0-9 as per tables).
- Used for ADDR, SUBR, MULR, DIVR, RMO, etc.
- Example: ADDR A, X (add register A into X) – opcode + r1=0, r2=1.

Format 3 (3 bytes) – relative/displacement addressing

op (6 bits) n i x b p e disp (12 bits)

- **Total bits:** $6+1+1+1+1+1+1+12 = 24$ bits (3 bytes).
- $e = 0$ distinguishes Format 3 from Format 4.
- n,i,x,b,p are flag bits (see addressing modes).
- disp is a 12-bit field.

Format 4 (4 bytes) – extended addressing (20-bit direct address)

op (6 bits) n i x b p e address (20 bits)

- **Total bits:** $6+1+1+1+1+1+1+20 = 32$ bits (4 bytes).
- $e = 1$.
- address is a full 20-bit memory address (0-1,048,575).

- Bits **b** and **p** are normally 0 (because direct addressing is used);

Addressing Modes

SIC/XE dramatically expands addressing capabilities.

Bit	Name	Function
n	indirect flag	Together with i selects addressing mode
i	immediate flag	
x	indexed flag	Adds (X) to target address after other calculations
b	base relative	Use register B for relative addressing
p	PC relative	Use register PC for relative addressing
e	format flag	0 = Format 3, 1 = Format 4

Addressing Mode Categories

A. Relative addressing (Format 3 only, **e=0**)

Mode	b	p	Target Address (TA)	Displacement range	Notes
Base relative	1	0	$TA = (B) + disp$	disp: 12-bit unsigned (0 to 4095)	Useful for accessing data near base register.
PC relative	0	1	$TA = (PC) + disp$	disp: 12-bit signed (-2048 to +2047)	For position-independent code.
Direct (Format 3)	0	0	$TA = disp$ (0-4095)	disp interpreted as unsigned 12-bit address	Only addresses 0-4095; rarely used.

B. Direct addressing (Format 4, **e=1**)

When **e=1**, normally **b = p = 0**. Then **TA = address** (20-bit field). This can address the full 1 MB memory directly.

C. Immediate and Indirect addressing (controlled by **n** and **i**)

n	i	Mode	Operand obtained
0	0	Simple addressing (compatibility mode)	TA is address of operand (same as standard SIC).
0	1	Immediate	TA itself is the operand value; no memory reference.
1	0	Indirect	Word at TA contains the address of the operand.
1	1	Simple (standard SIC/XE)	Same as <code>n=i=0</code> but with flags set to 1 (typical for assemblers).

Why SIC/XE Matters for System Software

- **Assemblers** must handle multiple formats, compute displacements for base/PC relative, and select Format 3 vs. 4 based on address range.
- **Compilers** can generate position-independent code (PC relative) and use immediate constants, reducing memory accesses.
- **Linkers** must process relocation for 20-bit addresses and compute 12-bit displacements.
- **Operating systems** use **SVC** for system calls and manage I/O channels.

Instruction Set Additions

All standard SIC instructions are retained. New categories:

Register load/store

- LDB, STB, LDS, STS, LDT, STT, LDF, STF

Register-to-register operations (Format 2)

- RMO (register move) – copies one register to another.
- ADDR, SUBR, MULR, DIVR – arithmetic on registers (not just A, can use A, X, L, B, S, T, etc.).

Floating-point arithmetic (using F register)

- ADDF, SUBF, MULF, DIVF – operate on 48-bit floats in memory or register F.

Supervisor call

- SVC – generates an interrupt to call operating system services.

I/O channel instructions (see next section)

- SIO, TIO, HIO

Other new instructions are listed in Appendix A.

Input/Output (I/O) Enhancements

Standard SIC's programmed I/O (TD, RD, WD) is still available.

New: I/O channels

- **Channels** allow the CPU to start an I/O operation and continue executing other instructions while the channel transfers data independently.
- Instructions:
 - SIO (Start I/O) – initiates a channel program.

- TIO (Test I/O) – checks channel status.
 - HIO (Halt I/O) – stops a channel.
- This enables **overlap of computation and I/O**, improving system efficiency.

Feature	Standard SIC	SIC/XE
Memory size	32 KB (15-bit addresses)	1 MB (20-bit addresses)
Registers	5 (A, X, L, PC, SW)	+ B, S, T, F (48-bit)
Floating-point	None	48-bit float (ADDF, etc.)
Instruction formats	1 fixed (24 bits)	4 formats (1,2,3,4 bytes)
Addressing modes	Direct, indexed	+ base relative, PC relative, immediate, indirect
Relative addressing	No	Yes (B and PC)
Immediate operands	No	Yes (#)
Indirect addressing	No	Yes (@)
I/O	Programmed (TD/RD/WD)	+ Channel I/O (SIO/TIO/HIO)
Supervisor call	No	SVC
Register arithmetic	Only A with memory	ADDR, SUBR, etc. (Format 2)



Load and Store for New Registers

These instructions load or store the values of the new registers B, S, T, and F.

Mnemonic	Operands	Format	Opcode (Hex)	Meaning / Effect
LDB	m	3/4	68	Load the word from memory address m into the B (Base) register.
LDS	m	3/4	6C	Load the word from memory address m into the S (General working) register.
LDT	m	3/4	74	Load the word from memory address m into the T (General working) register.
LDF	m	3/4	70	Load a floating-point value (3 bytes) from memory address m into the F register.
STB	m	3/4	78	Store the word from the B register into memory address m.
STS	m	3/4	7C	Store the word from the S register into memory address m.
STT	m	3/4	84	Store the word from the T register into memory address m.
STF	m	3/4	80	Store the floating-point value from the F register into memory address m.

Register-to-Register Operations

These instructions are highly efficient as they perform operations entirely within the CPU's registers.

Mnemonic	Operands	Format	Opcode (Hex)	Meaning / Effect
ADDR	r1, r2	2	90	ADD Register: $r2 \leftarrow (r2) + (r1)$
SUBR	r1, r2	2	94	SUBtract Register: $r2 \leftarrow (r2) - (r1)$
MULR	r1, r2	2	98	MULTiply Register: $r2 \leftarrow (r2) * (r1)$
DIVR	r1, r2	2	9C	DIVide Register: $r2 \leftarrow (r2) / (r1)$
COMPR	r1, r2	2	A0	COMPare Register: Compare (r1) and (r2) and set the Condition Code (CC).
RMO	r1, r2	2	AC	Register MOVE: $r2 \leftarrow (r1)$
SHIFTL	r1, n	2	A4	SHIFT Left the value in register r1 by n bits. Bits shifted out are lost; zeros are shifted in.
SHIFTR	r1, n	2	A8	SHIFT Right the value in register r1 by n bits; the sign bit is preserved.
TIXR	r1	2	B8	TIX with Register: $X \leftarrow X + 1$; Compare new (X) with (r1) and set CC.
CLEAR	r1	2	B4	Clear the contents of the register r1. $r1 \leftarrow 0$

Floating-Point Arithmetic

These instructions perform arithmetic on the 48-bit floating-point accumulator, register F.

Mnemonic	Operands	Format	Opcode (Hex)	Meaning / Effect
ADDF	m	3/4	58	ADD Floating-point: $F \leftarrow (F) + (m..m+5)$
SUBF	m	3/4	5C	SUBtract Floating-point: $F \leftarrow (F) - (m..m+5)$
MULF	m	3/4	60	MULTiply Floating-point: $F \leftarrow (F) * (m..m+5)$
DIVF	m	3/4	64	DIVide Floating-point: $F \leftarrow (F) / (m..m+5)$
COMPf	m	3/4	88	COMParE Floating-point: Compare (F) with memory operand and set CC.
FIX	-	1	C4	FIX: Convert the floating-point value in F to an integer and store it in A.
FLOAT	-	1	C0	FLOAT: Convert the integer in A to a floating-point value and store it in F.
NORM	-	1	C8	NORMALize: Normalize the floating-point value in the F register.

I/O Channel Instructions

These instructions manage DMA (Direct Memory Access) channels for efficient bulk I/O.

Mnemonic	Operands	Format	Opcode (Hex)	Meaning / Effect
SIO	-	1	F0	Start I/O channel. Starts the I/O channel number specified in register A.
TIO	-	1	F8	Test I/O channel. Tests the status of the channel specified in A.
HIO	-	1	F4	Halt I/O channel. Halts the I/O channel number specified in A.

RISC and CISC Machine Architecture

RISC and CISC are two approaches to processor design with different instruction handling strategies.

- RISC uses a small set of simple, fixed-length instructions and follows a load/store approach, enabling efficient and fast execution.
- CISC uses a larger set of complex, variable-length instructions that can perform multiple operations, often requiring multiple clock cycles.

Reduced Instruction Set Architecture (RISC)

RISC simplifies processor design by using a small, uniform set of instructions. Each instruction performs a basic operation (e.g., load, compute, store) and is designed to execute in a single clock cycle, enabling efficient pipelining and simpler hardware.

- **Fixed instruction length** (e.g., 32 bits for ARM, MIPS).
- **Load-store architecture** – arithmetic only on registers; memory access only via load/store.
- **Few addressing modes** (typically register indirect, immediate).
- **Large register set.**
- **Simple, single-cycle instructions** (pipeline friendly)

Characteristics of RISC

- Simpler instruction, hence simple instruction decoding.

- Fixed Instruction format
- It performs a smaller number of computer instructions. Thus, it has a high speed to operate.
- Instruction comes in the form of one word.
- More general-purpose registers for register-to-register operations.
- Simple Addressing Modes.
- The LOAD/STORE is the only instruction used to access the memory
- Optimized for pipelining due to uniform instruction size and simplicity.

Complex Instruction Set Architecture (CISC)

CISC reduces the number of instructions a program needs by using a large set of complex, variable-length instructions. A single instruction can perform multiple operations (e.g., load, compute, and store), which may take multiple clock cycles.

- **Variable instruction length** (e.g., x86: 1–15 bytes).
- **Memory-to-memory operations** – arithmetic can directly use memory operands.
- **Many addressing modes** (direct, indirect, indexed, base, etc.).
- **Fewer registers.**
- **Complex, multi-cycle instructions.**

Characteristics of CISC

- Complex instruction, hence complex instruction decoding.
- Instructions are larger than one-word size.
- Instruction may take more than a single clock cycle to get executed.
- Less number of general-purpose registers as operations get performed in memory itself.
- Complex Addressing Modes.

Example:

Suppose we have to add two 8-bit numbers:

- **CISC approach:** There will be a single command or instruction for this like ADD which will perform the task.
- **RISC approach:** Here programmer will write the first load command to load data in registers then it will use a suitable operator and then it will store the result in the desired location.

So, add operation is divided into parts i.e. load, operate, store due to which RISC programs are longer and require more memory to get stored but require fewer transistors due to less complex command.

Feature	Standard SIC	RISC	CISC
Instruction length	Fixed (24 bits)	Fixed	Variable
Addressing modes	2 (direct, indexed)	Few (typically 2-3)	Many
Arithmetic operands	Memory-to-register (ADD from memory to A)	Register-to-register (load-store)	Memory-to-memory / register-to-register
Register count	5 (small)	Many (≥ 16)	Few (≤ 8 typically)

RISC	CISC
Small and simple instruction set	Large and complex instruction set
Fixed-length instructions	Variable-length instructions
Usually 1 cycle per instruction	Multiple cycles per instruction
More general-purpose registers	Fewer registers
Simple and limited addressing modes	Complex and many addressing modes
Larger code size (more instructions)	Smaller code size (fewer instructions)
Simple hardware, easier pipelining	Complex hardware, harder pipelining
Examples: ARM, RISC-V, MIPS	Examples: x86, Intel 80386

Feature	SIC/XE	RISC	CISC
Instruction length	Variable (1,2,3,4 bytes)	Fixed	Variable
Addressing modes	Many (direct, indexed, indirect, immediate, base relative, PC relative)	Few	Many
Arithmetic	Memory-to-register (still <code>ADD M</code>) <i>but adds</i> register-to-register (<code>ADDR r1, r2</code>)	Load-store	Mixed
Registers	9 (A,X,L,B,S,T,F,PC,SW) – moderate	Large	Small

SIC Program

	LDA	FIVE	LOAD CONSTANT 5 INTO REGISTER A
	STA	ALPHA	STORE IN ALPHA
	LDCH	CHARZ	LOAD CHARACTER 'Z' INTO REGISTER A
	STCH	C1	STORE IN CHARACTER VARIABLE C1
	.		
	.		
	.		
ALPHA	RESW	1	ONE-WORD VARIABLE
FIVE	WORD	5	ONE-WORD CONSTANT
CHARZ	BYTE	C 'Z'	ONE-BYTE CONSTANT
C1	RESB	1	ONE-BYTE VARIABLE

(a)

	LDA	#5	LOAD VALUE 5 INTO REGISTER A
	STA	ALPHA	STORE IN ALPHA
	LDA	#90	LOAD ASCII CODE FOR 'Z' INTO REG A
	STCH	C1	STORE IN CHARACTER VARIABLE C1
	.		
	.		
	.		
ALPHA	RESW	1	ONE-WORD VARIABLE
C1	RESB	1	ONE-BYTE VARIABLE

(b)

Figure 1.2 Sample data movement operations for (a) SIC and (b) SIC/XE.

	LDA	ALPHA	LOAD ALPHA INTO REGISTER A
	ADD	INCR	ADD THE VALUE OF INCR
	SUB	ONE	SUBTRACT 1
	STA	BETA	STORE IN BETA
	LDA	GAMMA	LOAD GAMMA INTO REGISTER A
	ADD	INCR	ADD THE VALUE OF INCR
	SUB	ONE	SUBTRACT 1
	STA	DELTA	STORE IN DELTA
	.		
	.		
	.		
ONE	WORD	1	ONE-WORD CONSTANT
.			ONE-WORD VARIABLES
ALPHA	RESW	1	
BETA	RESW	1	
GAMMA	RESW	1	
DELTA	RESW	1	
INCR	RESW	1	

(a)

	LDS	INCR	LOAD VALUE OF INCR INTO REGISTER S
	LDA	ALPHA	LOAD ALPHA INTO REGISTER A
	ADDR	S,A	ADD THE VALUE OF INCR
	SUB	#1	SUBTRACT 1
	STA	BETA	STORE IN BETA
	LDA	GAMMA	LOAD GAMMA INTO REGISTER A
	ADDR	S,A	ADD THE VALUE OF INCR
	SUB	#1	SUBTRACT 1
	STA	DELTA	STORE IN DELTA
	.		
	.		
	.		
.			ONE WORD VARIABLES
ALPHA	RESW	1	
BETA	RESW	1	
GAMMA	RESW	1	
DELTA	RESW	1	
INCR	RESW	1	

(b)

Figure 1.3 Sample arithmetic operations for (a) SIC and (b) SIC/XE.

	LDX	ZERO	INITIALIZE INDEX REGISTER TO 0
MOVECH	LDCH	STR1,X	LOAD CHARACTER FROM STR1 INTO REG A
	STCH	STR2,X	STORE CHARACTER INTO STR2
	TIX	ELEVEN	ADD 1 TO INDEX, COMPARE RESULT TO 11
	JLT	MOVECH	LOOP IF INDEX IS LESS THAN 11
	.		
	.		
	.		
STR1	BYTE	C'TEST STRING'	11-BYTE STRING CONSTANT
STR2	RESB	11	11-BYTE VARIABLE
.			ONE-WORD CONSTANTS
ZERO	WORD	0	
ELEVEN	WORD	11	

(a)

	LDT	#11	INITIALIZE REGISTER T TO 11
	LDX	#0	INITIALIZE INDEX REGISTER TO 0
MOVECH	LDCH	STR1,X	LOAD CHARACTER FROM STR1 INTO REG A
	STCH	STR2,X	STORE CHARACTER INTO STR2
	TIXR	T	ADD 1 TO INDEX, COMPARE RESULT TO 11
	JLT	MOVECH	LOOP IF INDEX IS LESS THAN 11
	.		
	.		
	.		
STR1	BYTE	C'TEST STRING'	11-BYTE STRING CONSTANT
STR2	RESB	11	11-BYTE VARIABLE

(b)

Figure 1.4 Sample looping and indexing operations for (a) SIC and (b) SIC/XE.